TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS · CAPÍTULO 5

PostgreSQL numa API com Express

O banco da Loja Conecta ganha um servidor: conectar, montar o banco por script e fazer o CRUD em SQL

```
const { rows } = await pool.query(
  'SELECT * FROM produtos WHERE cod_produto = $1', [10]
);
```

PROFESSOR
Tarik Ponciano

CAPÍTULO 5 • BANCO DE DADOS

Roteiro

01 **Conexão**
O JavaScript conversa com o banco: pg, .env e a primeira consulta

02 **Setup**
O banco montado por um script: o .sql do Capítulo 4, rodado pelo Node

03 **CRUD**
A API de produtos: cada rota manda um SQL ao banco

04 **Regras na API**
FK, CHECK e UNIQUE viram respostas HTTP claras

★ **Consolidação**
O desafio dos pedidos: JOIN e a tabela N:N numa API

NESTE MATERIAL

Exemplo explicado
O professor monta a API da Loja Conecta: produtos, do SELECT ao DELETE.

Prática
Você implementa categorias e clientes, no mesmo projeto.

Código testado
Rodado no PostgreSQL e no Express 5, com a saída do terminal.

ONDE ESTAMOS

Dois caminhos que se encontram

CAPÍTULOS 3 E 4 • BANCO

SQL na Query Tool

A **Loja Conecta**, com FK, CHECK e JOIN. Quem mandava o SQL era você, no pgAdmin.

BACK-END • NODE.JS

Rotas com Express

GET, POST, PUT e DELETE, com os dados numa lista em memória ou num arquivo JSON.

CAPÍTULO 5 • JUNTAR

A API grava no banco

As rotas mandam SQL ao PostgreSQL. As regras do Capítulo 4 protegem a API também.

O que é novo

Só uma peça: a biblioteca **pg**, que leva o SQL do JavaScript até o PostgreSQL. O SQL você já sabe; as rotas também.

ONDE VAMOS CHEGAR

Uma API que responde com dados do banco

NO THUNDER CLIENT

```
GET http://localhost:3000/produtos/10

200 OK
{
  "cod_produto": 10,
  "nome_produto": "Notebook Lume 14",
  "preco": 3499,
  "estoque": 5,
  "cod_categoria": 1,
  "nome_categoria": "Informática"
}
```

O CAMINHO ATÉ AQUI

1 • Conectar
db.js e o .env: o Node entra no banco

2 • Montar
npm run setup: o banco nasce de um script

3 • Servir
server.js: o CRUD de produtos em SQL

4 • Proteger
as regras do banco viram respostas HTTP

Repare

nome_categoria não existe na tabela produtos: veio de um **LEFT JOIN** com categorias, feito pela API.

SEÇÃO 01

Conexão

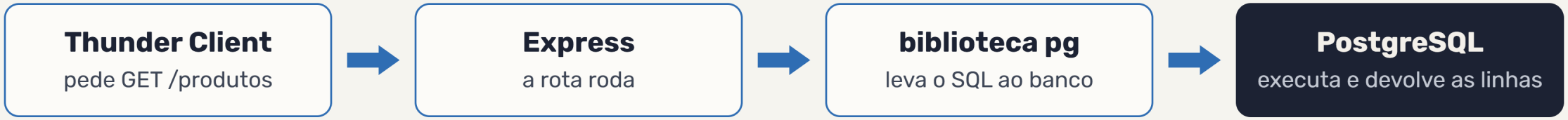
01

O JavaScript conversa com o PostgreSQL: biblioteca pg, dados de conexão e a primeira consulta.

01 • CONEXÃO

O Node é mais um cliente do banco

No Capítulo 3: o PostgreSQL é um **servidor** que espera conexões, e o pgAdmin é um **cliente**. Um programa em Node pode ser outro cliente, do mesmo jeito.



Quem manda o SQL

Antes: você, na Query Tool. Agora: o seu programa, a cada pedido que chega na API.

Quem cobra as regras

Sempre o banco. As FKs e os CHECKs do Capítulo 4 valem para o pgAdmin e para a API.

01 • CONEXÃO • PROJETO NOVO

Preparando o projeto

NO TERMINAL

```
# um projeto novo, como no Back-end
mkdir api-loja
cd api-loja
npm init -y
npm pkg set type=module
# o servidor e a biblioteca do PostgreSQL
npm install express pg
```

express
o servidor, o mesmo do Back-end

pg
a biblioteca que conversa com o PostgreSQL (node-postgres)

COMO FICA A PASTA

```
api-loja/
├── banco/
│   └── loja.sql      o script do banco
├── .env              dados de conexão + senha
├── .env.example      o modelo, sem senha
├── db.js             a conexão
├── setup.js          monta o banco
├── server.js         a API
└── package.json
```

Projeto do professor

Pronto e testado no GitHub da disciplina: **Banco de Dados Relacional / Capitulo 5 - API Loja Conecta**.

01 • CONEXÃO

Os mesmos cinco dados do pgAdmin

Para conectar no pgAdmin, você informou host, porta, banco, usuário e senha. A biblioteca pg pede exatamente esses cinco.

Dado	No computador	Na Aiven	No código
host	localhost	pg-…aivencloud.com	host
porta	5432	a do painel (ex.: 12691)	port
banco	loja	loja	database
usuário	postgres	avnadmin	user
senha	a da instalação	a do painel	password

Aiven

A conexão com a nuvem é criptografada (SSL). No código, ela pede mais uma linha: **ssl: { rejectUnauthorized: false }**.

01 · CONEXÃO

A senha fica fora do código: o arquivo .env

```
# .env · os SEUS dados (não vai para o GitHub)
DB_HOST=localhost
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=sua_senha
DB_NAME=loja
DB_ADMIN_NAME=postgres
DB_SSL=false
PORT=3000
```

NO PACKAGE.JSON

```
"scripts": {
  "setup": "node --env-file=.env setup.js",
  "dev": "node --watch --env-file=.env server.js"
}
```

.env

Os SEUS dados, com a senha. Está no .gitignore: **não vai para o GitHub**.

.env.example

O modelo, sem senha. Vai para o GitHub: quem clonar copia e preenche.

--env-file=.env

O Node lê o arquivo ao iniciar. No código, cada linha vira **process.env.NOME**.

npm run dev

Roda o script do package.json: ninguém precisa decorar o comando.

01 · CONEXÃO · EXEMPLO EXPLICADO

db.js: a conexão num arquivo só

```
// db.js · a conexão com o PostgreSQL
import pg from 'pg';

const { Pool } = pg;

export const pool = new Pool({
  host: process.env.DB_HOST,
  port: Number(process.env.DB_PORT),
  database: process.env.DB_NAME,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  ssl: process.env.DB_SSL === 'true'
    ? { rejectUnauthorized: false }
    : false,
});
```

LINHA A LINHA

import pg from 'pg'
A biblioteca instalada com npm install pg.

new Pool({ … })
Abre conexões com os cinco dados do .env.

ssl
Só liga quando DB_SSL=true (Aiven).

export const pool
Os outros arquivos usam: **import { pool } from './db.js'**

01 • CONEXÃO

Pool ou Client?

POOL • VÁRIAS CONEXÕES

Para o servidor

Um conjunto de conexões prontas. Cada pool.query pega uma livre, usa e devolve. Aguenta muitos pedidos ao mesmo tempo.

```
import { pool } from './db.js';

await pool.query(sql);
await pool.query(sql);
```

CLIENT • UMA CONEXÃO

Para scripts curtos

Uma conexão só: você abre, usa e fecha. É o que o setup.js usa, na Seção 02.

```
const cliente = new pg.Client(dados);
await cliente.connect();
await cliente.query(sql);
await cliente.end();
```

Comparando

Pool são os caixas abertos de um supermercado: quem chega vai para o livre. Client é um caixa só, aberto para uma compra e fechado em seguida.

01 • CONEXÃO • ANATOMIA

Anatomia de uma consulta

```
const { rows } = await pool.query(
  'SELECT nome_produto, preco FROM produtos WHERE cod_categoria = $1',
  [3]
);
```

1

O resultado

rows: as linhas. Também vem rowCount: quantas.

2

pool.query

Manda o SQL e espera a resposta: por isso o **await**.

3

O SQL

O mesmo da Query Tool, com **$1** no lugar do valor.

4

Os valores

Num array, na ordem: o 1º vai no $1, o 2º no $2…

Por que await?

Ir ao banco leva tempo. O await espera a resposta antes da próxima linha, como no readFile e no Inquirer. Quem usa await fica numa função **async**.

01 • CONEXÃO

O que volta: rows e rowCount

NA QUERY TOOL

nome_produto	preco
Capa para celular	49.90
Mouse sem fio	89.90
Fone Bluetooth	199.90



NO JAVASCRIPT

```
resultado.rows = [
  { nome_produto: 'Capa para celular', preco: 49.9 },
  { nome_produto: 'Mouse sem fio', preco: 89.9 },
  { nome_produto: 'Fone Bluetooth', preco: 199.9 }
]

resultado.rowCount = 3
```

Um array de objetos

Cada linha vira um objeto { coluna: valor }. forEach, map e filter funcionam.

Uma linha só?

rows[0]. Nenhuma? rows.length === 0: na API, isso vira 404.

rowCount

No UPDATE e no DELETE: quantas linhas mudaram. Zero: o código não existia.

01 • CONEXÃO • SEGURANÇA

Valores sempre como $1

EVITE • JUNTAR TEXTO

```
pool.query(
  'SELECT * FROM produtos WHERE cod_produto = '
    + req.params.id
);
```

Se alguém pedir **/produtos/1 OR 1=1**, o banco recebe outro comando e devolve tudo. Isso é **injeção de SQL**.

SEMPRE • PARÂMETRO

```
pool.query(
  'SELECT * FROM produtos WHERE cod_produto = $1',
  [req.params.id]
);
```

O pg manda o comando e o valor **separados**. O banco trata o valor sempre como dado, nunca como SQL: "1 OR 1=1" não é um número, e a consulta é recusada.

VÁRIOS VALORES: $1, $2… NA MESMA ORDEM DO ARRAY

```
pool.query(
  'SELECT * FROM produtos WHERE preco > $1 AND cod_categoria = $2',
  [500, 1]
);
```

01 · CONEXÃO · EXEMPLO EXPLICADO

A primeira consulta, sem servidor

```
// primeira-consulta.js
import { pool } from './db.js';

const resultado = await pool.query(
  'SELECT current_database() AS banco');
console.log(resultado.rows[0]);

const { rows } = await pool.query(
  'SELECT * FROM categorias ORDER BY cod_categoria');
console.table(rows);

await pool.end();
```

```
$ node --env-file=.env primeira-consulta.js
{ banco: 'loja' }
```

(index)	cod_categoria	nome_categoria
0	1	'Informática'
1	2	'Celulares'
2	3	'Acessórios'
3	4	'Games'

pool.end()

Fecha as conexões. Num script que termina, sem ele o terminal fica parado. No servidor, não feche: ele fica no ar.

01 • CONEXÃO • EXEMPLO EXPLICADO

Um JOIN do Capítulo 4, sem mudar nada

```
const pedidos = await pool.query(`
  SELECT p.cod_pedido, c.nome_cliente, p.status
  FROM pedidos p
  JOIN clientes c ON c.cod_cliente = p.cod_cliente
  WHERE c.cidade_cliente = $1
  ORDER BY p.data_pedido DESC
`, ['Recife']);

console.table(pedidos.rows);

const abertos = pedidos.rows.filter((p) => p.status === 'aberto');
console.log(`${abertos.length} em aberto`);
```

Crase: SQL em várias linhas

Do jeito que você escreve na Query Tool.

O JOIN é o mesmo

Pedidos com o nome do cliente: Capítulo 4, Seção 04.

$1 no WHERE

A cidade vai como parâmetro.

filter nas linhas

rows é um array: o que você sabe de coleções vale aqui.

Resultado

6 pedidos de Recife • 2 em aberto.

PRÁTICA 1 • 20 MIN • INDIVIDUAL

Conecte e consulte

a. Crie o projeto **api-loja**: npm init, type module, **npm install express pg**.

b. Crie o **.env** com os dados do SEU PostgreSQL (DB_NAME=loja, o banco do Capítulo 4) e o **.env.example**, sem a senha.

c. Crie o **db.js** com o Pool.

d. Crie o **consultas.js**: mostre o banco e o usuário conectados. Rode: node --env-file=.env consultas.js

e. Acrescente: os produtos da categoria **1**, com **$1**. Mostre um por linha, com forEach: nome e preço.

DEU CERTO SE APARECER

```
{ banco: 'loja', usuario: 'postgres' }
Notebook Lume 14 3499.00
Monitor 24 pol 899.90
```

Errou a senha de propósito? Leia a mensagem: ela diz o que houve.

PENSE E RESPONDA Some os dois preços: rows[0].preco + rows[1].preco. Deu 4398.9? O que apareceu, e por quê?

01 • CONEXÃO • PARA SABER

Números e datas vindos do banco

Tipo no banco	O pg entrega	Por quê	Com o ajuste
numeric(8,2)	"3499.00" (texto)	numeric é exato; o number do JS, não	3499
date	2026-09-02T03:00:00.000Z	vira um horário, com fuso	"2026-09-02"

NO DB.JS, LOGO DEPOIS DO IMPORT

```
const { Pool, types } = pg;

// numeric → número: 3499.00 vem como 3499, e não "3499.00"
types.setTypeParser(types.builtins.NUMERIC, (valor) ⇒ Number(valor));

// date → '2026-09-02', igual ao pgAdmin, sem fuso horário
types.setTypeParser(types.builtins.DATE, (valor) ⇒ valor);
```

Para preços de uma loja, number resolve. Em sistema de banco (dinheiro de verdade, muitos centavos), mantenha o texto e converta com cuidado.

SEÇÃO 02

Setup

02

O banco montado por um script: o Node cria o banco e roda o .sql do Capítulo 4.

02 • SETUP

Do F5 ao npm run setup

Passo	No Capítulo 4 • pgAdmin	No Capítulo 5 • setup.js
1	Query Tool no banco postgres	Conecta no banco postgres (Client)
2	CREATE DATABASE loja;	CREATE DATABASE loja, só se ainda não existir
3	Refresh e Query Tool no banco loja	Conecta no banco loja
4	Open File: o script .sql	readFile: lê banco/loja.sql
5	F5: roda tudo	loja.query(script): manda tudo de uma vez
6	Conferir as contagens	Mostra as contagens no terminal

Por que um script

Um colega clona o projeto, preenche o .env e roda **npm run setup**: o banco dele fica igual ao seu. Errou os dados nos testes? Rode de novo e tudo volta ao início.

02 · SETUP

O script: o do Capítulo 4, com uma mudança

```
-- Capítulo 4: o código vinha pronto nos dados
cod_produto integer PRIMARY KEY,

-- Capítulo 5: o banco gera o código
cod_produto integer GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
```

Por quê?

Agora quem cadastra é a API, e ela não sabe qual é o próximo código. O banco gera.

BY DEFAULT, não ALWAYS

Os dados do Capítulo 4 entram com os códigos de lá: 10, 11, 100… O BY DEFAULT aceita.

O resto é igual

DROP filhos → pais, CREATE pais → filhos, INSERT. As mesmas FKs e CHECKs, com os mesmos nomes.

Onde está

banco/loja.sql, no projeto do GitHub. Ele também roda inteiro no pgAdmin (F5).

02 · SETUP · ATENÇÃO

O contador não viu os códigos "na mão"

```
-- os dados entraram com os códigos 1, 2, 3 e 4; o contador ainda está no 1
INSERT INTO categorias (nome_categoria) VALUES ('Áudio');
ERROR: duplicate key value violates unique constraint "categorias_pkey"
DETAIL: Key (cod_categoria)=(1) already exists.
```

O problema do serial, no Capítulo 4: os INSERTs informaram o código e o contador ficou no 1. A solução vai no fim do script:

```
-- no fim do loja.sql: cada contador continua depois do maior código dos dados
ALTER TABLE categorias   ALTER COLUMN cod_categoria RESTART WITH 5;
ALTER TABLE produtos     ALTER COLUMN cod_produto   RESTART WITH 17;
ALTER TABLE clientes     ALTER COLUMN cod_cliente   RESTART WITH 7;
ALTER TABLE pedidos      ALTER COLUMN cod_pedido    RESTART WITH 108;
ALTER TABLE itens_pedido ALTER COLUMN cod_item      RESTART WITH 12;
```

Resultado: o primeiro produto cadastrado pela API recebe o **17**, o maior código dos dados + 1.

02 · SETUP

O setup conecta em outro banco: os dados viram uma função

```
// db.js · os dados de conexão viram uma função
export function dadosDeConexao(banco = process.env.DB_NAME) {
  return {
    host: process.env.DB_HOST,
    port: Number(process.env.DB_PORT),
    database: banco,
    user: process.env.DB_USER,
    password: process.env.DB_PASSWORD,
    ssl: process.env.DB_SSL === 'true'
      ? { rejectUnauthorized: false } : false,
  };
}

// o servidor continua igual: sem argumento, usa o DB_NAME do .env
export const pool = new Pool(dadosDeConexao());
```

O problema

Para criar o banco loja, o setup conecta antes no banco **postgres**: a "Query Tool no banco postgres".

A função

Recebe o nome do banco e devolve os cinco dados.

Sem argumento

Usa o DB_NAME do .env: o pool do servidor não muda.

setup.js • 1 de 2: criar o banco

```
// setup.js · parte 1: criar o banco, se ainda não existir
import { readFile } from 'node:fs/promises';
import path from 'node:path';
import pg from 'pg';
import { dadosDeConexao } from './db.js';

const BANCO = process.env.DB_NAME;
const servidor = new pg.Client(dadosDeConexao('postgres'));
await servidor.connect();

const { rowCount } = await servidor.query(
  'SELECT datname FROM pg_database WHERE datname = $1', [BANCO]);

if (rowCount === 0) {
  await servidor.query(`CREATE DATABASE ${BANCO}`);
}
await servidor.end();
```

new pg.Client
Uma conexão só, no banco postgres.

pg_database
A lista de bancos do servidor: a pasta Databases do pgAdmin.

rowCount === 0
Não existe? Cria. Já existe? Segue.

CREATE DATABASE sem $1
O nome não aceita parâmetro: vai no texto. Por isso vem do .env, nunca do usuário.

02 · SETUP · EXEMPLO EXPLICADO

setup.js • 2 de 2: rodar o script e conferir

```
// setup.js · parte 2: rodar o script e conferir
const loja = new pg.Client(dadosDeConexao(BANCO));
await loja.connect();
loja.on('notice', (aviso) ⇒ console.log('aviso:', aviso.message));

const caminho = path.join(import.meta.dirname, 'banco', 'loja.sql');
const script = await readFile(caminho, 'utf-8');
await loja.query(script);

const { rows } = await loja.query(`
  SELECT (SELECT count(*)::int FROM categorias) AS categorias,
         (SELECT count(*)::int FROM produtos)   AS produtos`);
console.table(rows);
await loja.end();
```

readFile
O .sql é só texto: o mesmo readFile do Back-end.

loja.query(script)
Tudo de uma vez, como o F5. Sem $1, o pg aceita vários comandos separados por ;

notice
Os avisos da aba Messages.

count(*)::int
O count vem como texto; ::int pede um inteiro.

02 · SETUP

npm run setup: o que aparece no terminal

```
$ npm run setup
1) Conectando no servidor localhost:5432 (banco postgres)...
2) Banco "loja" criado.
3) Rodando banco/loja.sql (DROP, CREATE e INSERT)...
   aviso do banco: table "itens_pedido" does not exist, skipping
   aviso do banco: table "pedidos" does not exist, skipping
   … (um aviso para cada tabela)
4) Conferência (esperado: 4 · 7 · 6 · 8 · 11):
┌─────────┬────────────┬──────────┬──────────┬──────────┬────────┐
│ (index) │ categorias │ produtos │ clientes │ pedidos  │ itens  │
├─────────┼────────────┼──────────┼──────────┼──────────┼────────┤
│ 0       │ 4          │ 7        │ 6        │ 8        │ 11     │
└─────────┴────────────┴──────────┴──────────┴──────────┴────────┘
Pronto! Banco "loja" montado.
```

Os avisos amarelos

Vêm do DROP TABLE IF EXISTS: só na primeira vez. Os mesmos da aba Messages.

Rode de novo

O passo 2 diz que o banco já existe, e as tabelas voltam ao início.

No pgAdmin

Refresh: o banco loja, as 5 tabelas, as FKs e os CHECKs.

Quando o setup não roda

Mensagem	Causa provável	O que fazer
node: .env: not found	Falta o arquivo .env	Copie o .env.example para .env
password authentication failed	Senha errada no .env	A mesma senha do pgAdmin
ECONNREFUSED	PostgreSQL desligado ou porta errada	Confira o serviço e o DB_PORT
database "postgres" does not exist	Na Aiven, o banco inicial é outro	DB_ADMIN_NAME=defaultdb
permission denied to create database	Usuário sem permissão	Use postgres (ou avnadmin)
syntax error at or near …	Erro no loja.sql	A mensagem diz o comando: corrija e rode de novo

O setup.js do GitHub mostra uma dica para cada um destes casos.

PRÁTICA 2 • 25 MIN • INDIVIDUAL

O banco por script

a. Copie o **banco/loja.sql** do projeto do GitHub para a sua pasta **banco/**.

b. No **db.js**, transforme os dados de conexão na função **dadosDeConexao**.

c. Escreva o **setup.js** (as duas partes) e o script **"setup"** no package.json.

d. Rode **npm run setup** duas vezes: a conferência mostra 4 • 7 • 6 • 8 • 11 nas duas.

e. No pgAdmin, cadastre uma categoria **sem informar o código**. Qual código ela recebeu?

VOLTE À PRÁTICA 1

Rode o consultas.js de novo, agora no banco montado pelo setup. Deu o mesmo resultado?

E depois do passo e: rode o setup mais uma vez. A categoria que você cadastrou ainda existe?

PENSE E RESPONDA O setup apagou as mudanças que você tinha feito no banco loja no Capítulo 4. Isso é um problema ou uma vantagem? Quando?

SEÇÃO 03

CRUD

03

A API de produtos: as rotas do Back-end, agora com um SQL em cada uma.

03 · CRUD

Cada rota, um comando SQL

Rota	SQL	Responde
GET /produtos	SELECT … LEFT JOIN categorias	200 e a lista
GET /produtos?categoria=3	o mesmo, com WHERE p.cod_categoria = $1	200 e a lista filtrada
GET /produtos/:id	SELECT … WHERE p.cod_produto = $1	200, ou 404
POST /produtos	INSERT … RETURNING *	201 e o produto criado
PUT /produtos/:id	UPDATE … SET … = COALESCE(…) RETURNING *	200, ou 404
DELETE /produtos/:id	DELETE … WHERE cod_produto = $1	204, ou 404

O que muda em relação ao Back-end

As rotas, os métodos e o JSON são os mesmos do CRUD em memória. Muda só onde os dados moram: no lugar de **find**, **push** e **splice**, um SQL.

server.js: a mesma base do Back-end

```
// server.js · a mesma base do Back-end
import express from 'express';
import { pool } from './db.js';

const app = express();
app.use(express.json());

// ... as rotas de produtos

const PORTA = process.env.PORT || 3000;
try {
  await pool.query('SELECT 1');   // o banco responde?
  app.listen(PORTA, () ⇒ console.log(`http://localhost:${PORTA}`));
} catch (erro) {
  console.error('Sem conexão com o banco:', erro.message);
  process.exit(1);
}
```

import { pool }

A conexão do db.js.

SELECT 1 antes do listen

Se o .env estiver errado, você descobre ao subir, e não no primeiro GET.

npm run dev

node --watch --env-file=.env server.js

Ler todos: GET /produtos

```
const SELECT_PRODUTOS = `
  SELECT p.cod_produto, p.nome_produto, p.preco, p.estoque,
         p.cod_categoria, c.nome_categoria
  FROM produtos p
  LEFT JOIN categorias c ON c.cod_categoria = p.cod_categoria`;

app.get('/produtos', async (req, res) => {
  try {
    const { rows } = await pool.query(
      `${SELECT_PRODUTOS} ORDER BY p.nome_produto`);
    res.json(rows);
  } catch (erro) {
    console.error(erro.message);
    res.status(500).json({ erro: 'Erro no banco de dados' });
  }
});
```

SELECT_PRODUTOS

O começo do SELECT, guardado numa constante: as duas rotas GET usam.

LEFT JOIN

Traz o nome da categoria. Com JOIN comum, o **Cabo HDMI** (sem categoria) sumiria: o "INNER quando a pergunta diz todos", do Capítulo 4.

res.json(rows)

A lista de linhas vira o JSON da resposta.

03 · CRUD · EXEMPLO EXPLICADO

Filtrar pela query string: ?categoria=3

```
app.get('/produtos', async (req, res) ⇒ {
  try {
    const { categoria } = req.query;
    let resultado;
    if (categoria) {
      resultado = await pool.query(
        `${SELECT_PRODUTOS} WHERE p.cod_categoria = $1 ORDER BY p.nome_produto`,
        [categoria]);
    } else {
      resultado = await pool.query(`${SELECT_PRODUTOS} ORDER BY p.nome_produto`);
    }
    res.json(resultado.rows);
  } catch (erro) { /* ... o mesmo catch */ }
});
```

Pode juntar texto aqui?

${SELECT_PRODUTOS} é texto FIXO do programa: pode. O valor do usuário (**categoria**) vai sempre no $1. Teste: ?categoria=3 → Capa, Fone e Mouse.

03 · CRUD · EXEMPLO EXPLICADO

Ler um: GET /produtos/:id

```
app.get('/produtos/:id', async (req, res) => {
  try {
    const { rows } = await pool.query(
      `${SELECT_PRODUTOS} WHERE p.cod_produto = $1`,
      [req.params.id]);

    if (rows.length === 0) {
      return res.status(404).json({ erro: 'Produto não encontrado' });
    }
    res.json(rows[0]);
  } catch (erro) { /* ... o mesmo catch */ }
});
```

O QUE MUDOU

find → WHERE … = $1
O filtro sai do JavaScript e vai para o SQL.

rows.length === 0
Lista vazia: esse código não existe. 404.

rows[0]
A consulta sempre devolve uma lista, mesmo com uma linha só.

Teste
/produtos/10 → 200, Notebook Lume 14 • /produtos/999 → 404

PRÁTICA 3 • 20 MIN • INDIVIDUAL

Leia categorias e clientes

a. **GET /categorias**: todas, em ordem alfabética.

b. **GET /categorias/:id**: uma; 404 se não existir.

c. **GET /clientes**, com o filtro **?cidade=Recife** (modelo: o ? categoria= dos produtos).

d. **GET /clientes/:id**: um; 404 se não existir.

TESTES NO THUNDER CLIENT

Requisição	Esperado
GET /categorias	200 • 4, Acessórios primeiro
GET /categorias/9	404
GET /clientes?cidade=Recife	200 • Lívia, Mariana e Paula
GET /clientes/6	200 • Gustavo Ramos

PENSE E RESPONDA GET /clientes?cidade=recife, com r minúsculo, devolve uma lista vazia. Por quê? E sem erro nenhum!

03 · CRUD · EXEMPLO EXPLICADO

Criar: POST com INSERT … RETURNING

```
app.post('/produtos', async (req, res) ⇒ {
  try {
    const { nome_produto, preco, estoque, cod_categoria } = req.body ?? {};
    if (!nome_produto || preco ═══ undefined) {
      return res.status(400).json({ erro: 'Informe nome_produto e preco' });
    }
    const { rows } = await pool.query(
      `INSERT INTO produtos (nome_produto, preco, estoque, cod_categoria)
       VALUES ($1, $2, COALESCE($3, 0), $4)
       RETURNING *`,
      [nome_produto, preco, estoque, cod_categoria]);
    res.status(201).json(rows[0]);
  } catch (erro) { /* ... o mesmo catch */ }
});
```

Sem cod_produto
O banco gera: o 17.

COALESCE($3, 0)
Sem estoque? Usa 0.

RETURNING *
Devolve a linha criada.

req.body ?? {}
Sem corpo, nada quebra.

Atualizar: PUT com UPDATE e COALESCE

```
app.put('/produtos/:id', async (req, res) => {
  try {
    const { nome_produto, preco, estoque, cod_categoria } = req.body ?? {};
    const { rows } = await pool.query(
      `UPDATE produtos
       SET nome_produto  = COALESCE($1, nome_produto),
           preco         = COALESCE($2, preco),
           estoque       = COALESCE($3, estoque),
           cod_categoria = COALESCE($4, cod_categoria)
       WHERE cod_produto = $5
       RETURNING *`,
      [nome_produto, preco, estoque, cod_categoria, req.params.id]);
    if (rows.length === 0) {
      return res.status(404).json({ erro: 'Produto não encontrado' });
    }
    res.json(rows[0]);
  } catch (erro) { /* ... o mesmo catch */ }
});
```

COALESCE($1, nome_produto)
Usa o valor novo; se ele não veio (null), mantém o atual. É o ?? do SQL.

$5 = req.params.id
A posição no array decide: o 5º valor vai no $5.

rows.length === 0
Nenhuma linha atualizada: 404.

Teste
PUT /produtos/17 com {"preco": 229.9} → só o preço muda.

Remover: DELETE e o rowCount

```
app.delete('/produtos/:id', async (req, res) => {
  try {
    const { rowCount } = await pool.query(
      'DELETE FROM produtos WHERE cod_produto = $1',
      [req.params.id]);

    if (rowCount === 0) {
      return res.status(404).json({ erro: 'Produto não encontrado' });
    }
    res.status(204).send();
  } catch (erro) { /* ... o mesmo catch */ }
});
```

rowCount

Quantas linhas o DELETE apagou. Zero: o código não existia, 404.

204

Deu certo, e não há nada para mostrar: send() sem corpo.

Tente: DELETE /produtos/10

Dá erro! O notebook já foi vendido: a FK **fk_item_produto** não deixa apagar. Por enquanto o catch responde 500. A Seção 04 resolve.

03 · CRUD

Toda rota com try e catch

```
app.get('/produtos', async (req, res) ⇒ {
  try {
    // tudo o que fala com o banco
  } catch (erro) {
    console.error(erro.message);           // o motivo, no terminal
    res.status(500).json({ erro: 'Erro no banco de dados' });
  }
});
```

Banco fora do ar

Senha errada, PostgreSQL desligado, a internet caiu na Aiven.

SQL com erro

Nome de tabela errado, vírgula sobrando, coluna que não existe.

O banco recusou

Uma regra do Capítulo 4: FK, CHECK, UNIQUE, NOT NULL.

Teste

Troque **produtos** por **produto** no SQL e faça GET /produtos: 500, e o terminal mostra relation "produto" does not exist.

03 · CRUD

Testes no Thunder Client, na ordem

#	Requisição	Corpo (JSON)	Esperado
1	GET /produtos	—	200 • 7 produtos; Cabo HDMI com nome_categoria null
2	GET /produtos?categoria=3	—	200 • Capa, Fone e Mouse
3	GET /produtos/999	—	404
4	POST /produtos	{"nome_produto": "Teclado mecânico", "preco": 249.9, "cod_categoria": 1}	201 • cod_produto 17, estoque 0
5	PUT /produtos/17	{"preco": 229.9}	200 • só o preço mudou
6	POST /produtos	{"nome_produto": "Pendrive", "preco": 59.9}	201 • cod_produto 18
7	DELETE /produtos/18	—	204
8	GET /produtos/18	—	404

Depois de cada um, confira no pgAdmin. Reinicie o servidor e faça GET /produtos: **os dados continuam lá**.

PRÁTICA 4 • 30 MIN • INDIVIDUAL

O CRUD completo de categorias

a. **POST /categorias** com {"nome_categoria": "Áudio"}: 400 se faltar o nome; senão INSERT … RETURNING * e 201.

b. **PUT /categorias/:id**: troca o nome; 404 se não existir.

c. **DELETE /categorias/:id**: 204; 404 se não existir.

d. **try/catch** em todas as rotas de categorias.

TESTES (BANCO RECÉM-MONTADO)

Requisição	Esperado
POST /categorias • "Áudio"	201 • cod_categoria 5
PUT /categorias/5 • "Áudio e Som"	200
DELETE /categorias/5	204
DELETE /categorias/9	404

PENSE E RESPONDA Agora tente POST /categorias com "Games", que já existe, e DELETE /categorias/1. O que a API responde? Está certo?

SEÇÃO 04

Regras na API

04

As restrições do Capítulo 4 continuam no banco. A API transforma cada recusa numa resposta clara.

04 • REGRAS NA API

O banco continua recusando, agora pela API

Pedido	Regra do Capítulo 4 que barrou	A API responde hoje
POST /produtos com "preco": -5	ck_preco • CHECK	500 • Erro no banco de dados
POST /produtos com "cod_categoria": 99	fk_prod_categoria • FOREIGN KEY	500 • Erro no banco de dados
POST /categorias com "Games"	uq_categoria_nome • UNIQUE	500 • Erro no banco de dados
DELETE /produtos/10	fk_item_produto • FOREIGN KEY	500 • Erro no banco de dados
GET /produtos/abc	o tipo integer da coluna	500 • Erro no banco de dados

O problema

O banco fez o trabalho certo: barrou o dado errado. Mas **500** diz "a culpa é do servidor", e quem errou foi o pedido. O certo é **400** ou **409**, com uma mensagem que explique.

04 • REGRAS NA API

O que vem dentro do erro

```
// o que o pg entrega no catch (POST com "preco": -5)
erro.code        // '23514'
erro.constraint  // 'ck_preco'
erro.detail      // 'Failing row contains (19, Teste, -5.00, 0, null).'
erro.message     // 'new row for relation "produtos" violates check constraint "ck_preco"'
```

QUAL ERRO

erro.code

Um código de 5 caracteres para cada tipo de erro. A lista oficial está na documentação do PostgreSQL (Appendix A).

QUAL REGRA

erro.constraint

O nome que demos à regra no CREATE TABLE. Por isso, no Capítulo 4, toda regra ganhou nome.

QUAL DADO

erro.detail

O dado que causou o erro: Key (cod_categoria)=(99) is not present…

04 • REGRAS NA API

Cada erro do banco, um status HTTP

Restrição	erro.code	Status	Exemplo na loja
FOREIGN KEY, ao cadastrar	23503	400	produto com cod_categoria 99
FOREIGN KEY, ao excluir	23503	409	excluir o produto 10, já vendido
UNIQUE	23505	409	categoria "Games" de novo
CHECK	23514	400	preço -5
NOT NULL	23502	400	"preco": null
tipo errado	22P02	400	GET /produtos/abc
qualquer outro	—	500	banco desligado, tabela com nome errado

400 • Bad Request
O pedido veio errado: corrija o dado e tente de novo.

409 • Conflict
O pedido bate de frente com o que já existe no banco.

500 • Server Error
Falha do servidor: o cliente não tem o que corrigir.

erros.js: uma função para todas as rotas

```
export function responderErro(req, res, erro) {
  const resposta = { erro: '', regra: erro.constraint, detalhe: erro.detail };
  switch (erro.code) {
    case '23503': // FOREIGN KEY
      if (req.method === 'DELETE') {
        resposta.erro = 'Não é possível excluir: existem registros ligados a este.';
        return res.status(409).json(resposta);
      }
      resposta.erro = 'O código informado não existe na tabela de origem.';
      return res.status(400).json(resposta);
    case '23514': // CHECK
      resposta.erro = 'Valor fora da regra do banco.';
      return res.status(400).json(resposta);
    default:
      return res.status(500).json({ erro: 'Erro no banco de dados.' });
  }
}
```

04 · REGRAS NA API

No catch de cada rota: responderErro

```
// antes: tudo virava 500
} catch (erro) {
  console.error(erro.message);
  res.status(500).json({ erro: 'Erro no banco de dados' });
}
```

```
// agora: o erro do banco decide
import { responderErro } from './erros.js';
// ...
} catch (erro) {
  responderErro(req, res, erro);
}
```

O MESMO PEDIDO, AGORA

```
POST /produtos   { "nome_produto": "Teste", "preco": -5 }
400 Bad Request
{
  "erro": "Valor fora da regra do banco.",
  "regra": "ck_preco",
  "detalhe": "Failing row contains (19, Teste, -5.00, 0, null)."
}
```

04 • REGRAS NA API

Teste cada regra: todos devem ser recusados

Requisição	Corpo (JSON)	Esperado
POST /produtos	{"nome_produto": "Teste", "preco": -5}	400 • ck_preco
POST /produtos	{"nome_produto": "Teste", "preco": 10, "cod_categoria": 99}	400 • fk_prod_categoria
PUT /produtos/17	{"estoque": -1}	400 • ck_estoque
DELETE /produtos/10	—	409 • fk_item_produto
POST /categorias	{"nome_categoria": "Games"}	409 • uq_categoria_nome
GET /produtos/abc	—	400 • formato inválido

Repare

O produto recusado por preço negativo gastou o código 19. Um INSERT recusado também usa um número do contador, como no Capítulo 4: buracos na numeração são normais.

04 • REGRAS NA API

Validar na API ou no banco? Nos dois

IF NA ROTA

Na API: o básico do pedido

Veio o nome? Veio o preço? Responde rápido, com uma mensagem simples, antes de ir ao banco.

FK • CHECK • UNIQUE

No banco: as regras do negócio

Preço ≥ 0, categoria que existe, e-mail sem repetir. Valem para a API, para o pgAdmin e para qualquer outro sistema.

Validar só na API?

Um INSERT feito no pgAdmin, ou por outro programa, passa por cima. A regra que mora no banco não tem como ser pulada: é ela que protege os dados.

PRÁTICA 5 • 35 MIN • INDIVIDUAL

Clientes, com as regras do banco

a. Crie o **erros.js** e troque o catch de produtos e categorias por **responderErro**.

b. Faça o CRUD de **clientes**: as 5 rotas; nome, e-mail e cidade obrigatórios no POST; PUT com COALESCE.

c. Rode **npm run setup** e faça os testes ao lado: cada um precisa dar o status e a regra indicados.

TESTES

Requisição	Esperado
POST /clientes • e-mail mariana@email.com	409 • uq_cliente_email
DELETE /clientes/1	409 • fk_ped_cliente
DELETE /clientes/6	204
DELETE /categorias/1	409 • fk_prod_categoria
POST /categorias • "Games"	409 • uq_categoria_nome

PENSE E RESPONDA DELETE /clientes/6 funcionou e DELETE /clientes/1 não. O que o Gustavo tem de diferente da Mariana?

DESAFIO • 50 MIN • EM DUPLA

Os pedidos da loja

a. **GET /pedidos**: com o nome do cliente (JOIN) e o filtro **?status=aberto**.

b. **GET /pedidos/:id**: o pedido, o cliente e os itens com o nome do produto e o **total_item** (qtd × preco_unit). O total do pedido: some no JavaScript, com **reduce**.

c. **POST /pedidos** com {"cod_cliente": 6}: a data e o status vêm dos DEFAULTs.

d. **PUT /pedidos/:id** com {"status": "enviado"}. Teste "pausado": 400 • ck_status.

e. **DELETE /pedidos/:id**: os itens saem junto (ON DELETE CASCADE).

f. **POST /pedidos/:id/itens** com {"cod_produto": 15, "qtd": 2}: busque o preço do produto no banco antes do INSERT.

DESAFIO • DEU CERTO SE…

O que a API dos pedidos devolve

```
GET /pedidos/100
{
  "cod_pedido": 100, "data_pedido": "2026-09-02", "status": "entregue",
  "cod_cliente": 1, "nome_cliente": "Mariana Costa", "cidade_cliente": "Recife",
  "itens": [
    { "cod_item": 1, "cod_produto": 10, "nome_produto": "Notebook Lume 14",
      "qtd": 1, "preco_unit": 3499, "total_item": 3499 },
    { "cod_item": 2, "cod_produto": 13, "nome_produto": "Fone Bluetooth",
      "qtd": 2, "preco_unit": 199.9, "total_item": 399.8 }
  ],
  "total": 3898.8
}
```

GET /pedidos/107
itens [] e total 0: o pedido sem itens.

Mesmo produto 2 vezes
409 • uq_item_pedido_produto

"qtd": 0
400 • ck_qtd

Item no pedido 999
400 • fk_item_pedido

FECHAMENTO

Erros comuns e como resolver

Sintoma	Causa provável	Solução
node: .env: not found	Não existe o arquivo .env	Copie o .env.example para .env
password authentication failed	Senha errada no .env	A mesma do pgAdmin
database "loja" does not exist	Subiu o servidor antes do setup	npm run setup
relation "produto" does not exist	Nome da tabela errado no SQL	É produtos, no plural
column reference "..." is ambiguous	JOIN sem apelido na coluna	p.cod_categoria (Capítulo 4)
bind message supplies 1 parameters...	Faltou valor no array	Um valor para cada $
A rota responde {} ou trava	Faltou o await	await pool.query, em rota async
self-signed certificate (Aiven)	Faltou o SSL	DB_SSL=true no .env

FECHAMENTO

Confira antes de chamar o professor

Conexão

☐ O .env tem a sua senha e **não** aparece no git status.

☐ O db.js exporta o pool; o servidor sobe sem erro.

☐ Todo valor do usuário vai como $1, $2.

Setup

☐ npm run setup roda duas vezes seguidas, sem erro.

☐ A conferência mostra 4 • 7 • 6 • 8 • 11.

☐ Cadastro pela API recebe o código 17 (IDENTITY + RESTART).

CRUD e regras

☐ Produtos, categorias e clientes com as 5 rotas.

☐ Status 200, 201, 204, 400, 404 e 409 conferidos.

☐ Cada regra do banco testada, com o nome na resposta.

BANCO DE DADOS · CONSULTA RÁPIDA

Resumo

pg
npm install pg
new Pool({ … })
export const pool

.env
senha fora do código
--env-file=.env
process.env.DB_…

Consulta
await pool.query(sql, [v])
{ rows, rowCount }
um item: rows[0]

Parâmetros
$1, $2 na ordem
nunca juntar texto
evita injeção

Setup
Client: uma conexão
readFile + query
roda quantas
vezes quiser

IDENTITY
o banco gera o código
BY DEFAULT + dados
RESTART WITH maior + 1

Criar e mudar
INSERT … RETURNING *
COALESCE($1, coluna)
sem linhas: 404

Remover
DELETE … $1
rowCount 0: 404
senão: 204

JOIN na API
LEFT JOIN: todos
apelido em toda coluna
reduce para totais

Erros do banco
erro.code
erro.constraint
erro.detail

Status
400: pedido errado
409: conflito
500: falha nossa

Regras
na API: o básico
no banco: o negócio
as duas juntas

BANCO DE DADOS • CAPÍTULO CONCLUÍDO

O banco ganhou uma API!

Entregue o CRUD de categorias e clientes e o desafio dos pedidos. Dúvidas? Discord aberto.

Discord
discord.gg/wt5CVZZWJs

Drive
drive.google.com/drive/folders/1h0l0DaPeAor7gnhKBUIIZ5n8lLRDNvUY

GitHub
github.com/TarikPonciano/Programador-de-Sistema-SENAC

PARA ESTUDAR

node-postgres.com • documentação da biblioteca pg

expressjs.com/pt-br • documentação do Express

postgresql.org/docs • IDENTITY e os códigos de erro (Appendix A)